



UPES ACM & ACM-W

100 Days of Code

Day: 7

UPES ACM & ACM-W 100 Days of Code Platform

FIFA World Cup — The Corrupted Goal Tracker

Story

The FIFA World Cup Analytics Department is responsible for maintaining live statistics from three major stadiums hosting the tournament's knockout matches. Every goal scored is recorded along with the exact minute it occurred, while a central tracking system maintains goal counts for every player competing for the prestigious Golden Boot Award.

As the semi-finals approach, FIFA officials discover alarming inconsistencies in the analytics dashboard. Goals scored in Stadium 3 are mysteriously appearing in Stadium 2's records. Match reports show seven goals, but the system reports a different total. Even worse, the Golden Boot leaderboard is displaying the wrong player at the top, causing confusion among commentators and fans worldwide.

The software still compiles and produces output, making the bugs difficult to detect. Hidden array-indexing mistakes, boundary errors, and incorrect data updates are silently corrupting tournament statistics across all three stadiums.

You have been appointed as FIFA's emergency software engineer. Your task is to investigate the source code, identify every bug, and restore the integrity of the World Cup analytics system before the final match begins.

What the Code Must Do

- Store goal timings for three stadiums using arrays.
- Track goals scored by each player.
- Correctly record goals in the intended stadium.
- Calculate the total tournament goals.
- Identify the top scorer accurately.
- Display all goals for each stadium.
- Avoid out-of-bounds array access.

Expected Output

Total Goals: 7

Top Scorer: Player 2

Stadium 0 goals: 15 42

Stadium 1 goals: 8 67

Stadium 2 goals: 23 55 80

Buggy Code

```
void addGoalToStadium2(int minute, int playerID) {
    if (goalCount[2] >= MAX_GOALS) {
        printf("Stadium 2 is full\n");
        return;
    }
    goals[1][goalCount[2]] = minute;
    playerGoals[playerID]++;
    goalCount[2]++;
}

int totalGoals() {
    int total = 0;
    for (int s = 0; s <= STADIUMS; s++) {
        total += goalCount[s];
    }
    return total;
}

int topScorer() {
    int maxG = 0, winner = 0;
    for (int p = 1; p < PLAYERS; p++) {
        if (playerGoals[p] > maxG) {
            maxG = playerGoals[p];
            winner = p;
        }
    }
    return winner;
}

void displayStadium(int stadium) {
    for (int i = 0; i <= goalCount[stadium]; i++) {
        printf("%d ", goals[stadium][i]);
    }
}
```

Constraints

- $1 \leq \text{STADIUMS} \leq 10$
- $1 \leq \text{MAX_GOALS} \leq 100$
- $1 \leq \text{PLAYERS} \leq 50$
- Use arrays only.
- Correct the existing implementation.
- Do not redesign the program or use additional data structures.